
Advanced RV32I Core with Cache Subsystem

v0.1

Author

Nguyen To Quoc Viet

Table of Contents

1 Scope

1.1 Core Scope

The core documented in this report is the currently implemented RV32I five-stage in-order processor. It uses a single-issue pipeline with the classic IF, ID, EX, MEM, and WB stages. The implemented instruction set is the RV32I base integer ISA, including integer arithmetic, logical operations, shifts, comparisons, load/store instructions, conditional branches, JAL, JALR, LUI, and AUIPC.

The core includes dynamic branch prediction using a 1024-entry Branch History Table and Branch Target Buffer. Branch and jump instructions are resolved in the EX stage, and mispredictions redirect the fetch PC through a registered feedback path. Data hazards are handled by forwarding from later pipeline stages and by inserting a stall for load-use hazards.

1.2 Cache Subsystem Scope

The cache subsystem connects the processor core to an external AXI4 memory interface. It contains a 4 KB direct-mapped instruction cache, a 4 KB two-way set-associative data cache, a 4-entry write buffer, and a bus arbiter shared by instruction fetches, data-cache refills, and write-buffer drains. Both caches use 16-byte cache lines and a 32-bit data path.

The write buffer decouples stores from the external memory path and also provides store-to-load forwarding for pending writes. Cache misses are serviced through the AXI4 master interface.

1.3 Boundary

This document covers only the implemented RV32I five-stage baseline and its cache subsystem. RV32M, floating-point, vector/IME extensions, CSR, exception handling, interrupts, privilege levels, MMU, virtual memory, and full cache coherency are outside the implemented baseline.

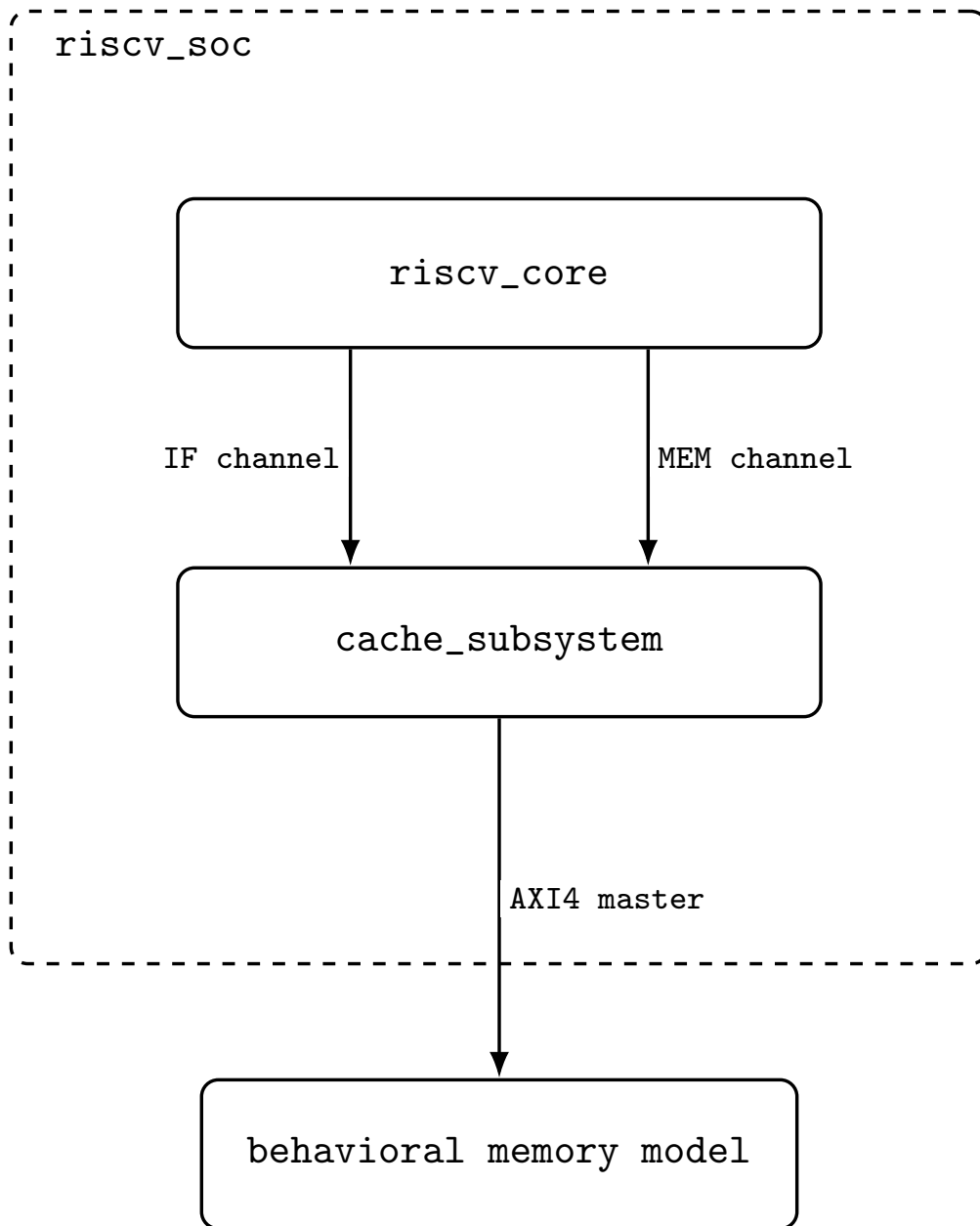
2 Source Repository

The source code of this project is maintained in the following GitHub repository:

`github.com/NguyenToQuocViet/Advanced-RISC-V-32-bit-SoC`

The architecture described in this document corresponds to the integrated five-stage RV32I baseline in the RTL source tree, especially the modules under `rtl/cpu/core/` and `rtl/cpu/cache/`.

3 Top-Level Architecture



Hình 1: Top-level organization of the implemented RV32I core and cache subsystem.

At the top level, the core-cache boundary is intentionally split into two independent channels. The IF channel carries instruction-fetch requests from the core to the cache subsystem, while the MEM channel carries load and store requests. This separation mirrors the pipeline structure: instruction traffic originates from the IF stage, whereas data-memory traffic originates from the MEM stage.

The IF channel carries the requested program counter and the returned instruction. The signal `if_req` indicates that the core requests the instruction at `if_pc`. The cache subsystem returns `if_instr`, `if_icache_valid`, and `if_icache_ready`, which determine whether the fetch stage can consume the instruction and advance the program counter.

The MEM channel carries the effective address, request type, store data, and byte write strobes. Loads and stores share `mem_addr` and `mem_req`. Stores additionally use `mem_we`, `mem_wdata`, and `mem_wstrb`. The cache subsystem returns `mem_rdata`, `mem_dcache_valid`, and `mem_dcache_ready`.

The cache subsystem is the only block that drives the external AXI4 master interface. At this level, AXI4 is treated as five grouped channels: read address, read data, write address, write data, and write response. Detailed AXI4 signal behavior is described with the cache subsystem rather than at the top-level boundary.

Interface	Direction	Main signals	Purpose
IF channel	Core ↔ cache	if_pc, if_req, if_instr, if_icache_ready, if_icache_valid	Instruction fetch
MEM channel	Core ↔ cache	mem_addr, mem_req, mem_we, mem_wdata, mem_wstrb, mem_rdata, mem_dcache_ready, mem_dcache_valid	Load/store access
AXI4 master	Cache ↔ memory	AR, R, AW, W, and B channels	External memory transactions

Bảng 1: Top-level interfaces between the core, cache subsystem, and memory.

- 4 Five-Stage Pipeline**
- 5 Core Modules**
- 6 Cache Subsystem**
- 7 Verification Surface**
- 8 Known Limitations**